Reporte de Correcciones Aplicadas — Práctica 1 UC3

> **Curso:** Ciberseguridad Aplicada · **Unidad de Competencia 3** · **ITSON Práctica:** P1 — Seguridad en APIs (BOLA · Mass Assignment · Validación estricta) · **Equipo:** Equipo Hector — Hector Andres Lagarda Alcantar (204733) · **Fecha:** 11/11/2026

Tabla de correcciones

ID Hallazgo	Vulnerabilidad	Archivo(s) modificados	Cambio aplicado (resumen técnico)	Evidencia de verificación	Estado
P1-001	BOLA de lectura en `GET /api/pedidos/{id}` (API1:2023)	`service/PedidoService.java`, `controller/PedidoController.java`	Se agregó verificación de propiedad en servidor: `pedido.usuario.id == usuarioAutenticado.id`; si no, **403** (`AccesoDenegadoException`); sin `X-User-Id`, **401**	`PruebaBolaTest.usuario1_solicitaPedidoDeBeto_retorna403` y `sinEncabezadoXUserId_retorna401` en verde + captura Postman 01 (403) y 02 (401)	Corregy verifica
P1-002	BOLA de escritura en `POST /api/pedidos` (API1:2023)	`dto/PedidoCreacionDTO.java` (nuevo), `service/PedidoService.java`, `controller/PedidoController.java`	El dueño **siempre** se resuelve del encabezado `X-User-Id` (vía `UsuarioAutenticadoService`); el DTO de entrada no incluye `usuario`, por lo que un intento de inyectarlo es rechazado por el esquema estricto	Captura Postman 04 (400: campo `usuario` desconocido) + flujo legítimo 00 (201 con `nombreUsuario` del autenticado)	Corregy verifica
P1-003	Mass Assignment en `POST /api/usuarios` (API3:2023 BOPLA)	`dto/UsuarioRegistroDTO.java`, `controller/UsuarioController.java`	DTO de entrada sin `id`; el campo `rol` se declara `WRITE_ONLY` (Jackson lo acepta pero el servidor lo ignora) y el rol lo asigna el servidor (siempre `USER`)	`PruebaMassAssignmentTest.rolAdminEnJson_seIgnoraYSeAlmacenaUSER` en verde + captura Postman 05 (201 con `"rol":"USER"`)	Corregy verifica
P1-004	Exposición de contraseñas en respuestas JSON (API3/API8:2023)	`model/Usuario.java` (`@JsonIgnore` en password), `dto/UsuarioRespuestaDTO.java` y `dto/PedidoRespuestaDTO.java` (nuevos)	Las respuestas usan DTOs de salida sin `password`; la entidad nunca se serializa completa	`PruebaBolaTest.usuario1_obtieneSuPropioPedido_retorna200` (respuesta con `nombreUsuario`, sin password) + captura Postman 00 de validación	Corregy verifica
P1-005	Validación inadecuada de esquemas de entrada (API8:2023)	`pom.xml` (agrega `spring-boot-starter-validation`), DTOs con `@Valid` + restricciones, `config/GlobalExceptionHandler.java`	Bean Validation con `@NotBlank`, `@Email`, `@Size`, `@Positive`, `@DecimalMax`; errores 400 centralizados con un mensaje por campo, sin stacktrace	`PruebaValidacionTest.emailInvalido_retorna400ConMensaje`, `passwordCorto_retorna400ConMensaje`, `nombreVacio_retorna400ConMensaje` en verde + capturas Postman 06 y 07 (400 con mensajes)	Corregy verifica
—	Esquema estricto de deserialización (complemento de P1-003 y P1-005)	`application.properties`	`spring.jackson.deserialization.fail-on-unknown-properties=true`: cualquier campo desconocido se rechaza con **400**	`PruebaMassAssignmentTest.campoDesconocidoEnJson_retorna400` en verde	Corregy verifica

Detalle de la corrección

P1-001 — Verificación de propiedad del recurso (anti-BOLA de lectura)

Antes (fragmento original):

```
public Optional<Pedido> obtenerPorId(Long id) {
    // ⚠️ VULNERABLE (BOLA): se busca el pedido por id SIN verificar que
    // pertenezca al usuario autenticado.
    return pedidoRepository.findById(id);
}
```

Después (fragmento corregido):

```
public Pedido obtenerDelUsuario(Long id, Usuario usuario) {
    Pedido pedido = pedidoRepository.findById(id)
            .orElseThrow(() -> new RecursoNoEncontradoException("Pedido no encontrado: " + id));

    if (!pedido.getUsuario().getId().equals(usuario.getId())) {
        throw new AccesoDenegadoException("No tiene permisos para acceder al pedido solicitado");
    }
    return pedido;
}
```

¿Por qué esta corrección mitiga el hallazgo? La decisión de autorización ya no depende del identificador que envía el cliente, sino de una comparación realizada en el servidor contra el usuario autenticado (resuelto del encabezado `X-User-Id`). El pedido ajeno nunca sale de la capa de

servicio: se responde 403, y sin identidad 401.

P1-002 — Dueño del pedido siempre desde la sesión (anti-BOLA de escritura)

Antes: `POST /api/pedidos` enlazaba la entidad completa y el dueño salía del cuerpo: `"usuario": {"id": 2}`.

Después: el controlador recibe `PedidoCreacionDTO` (sin campo `usuario`) y el dueño se obtiene del encabezado:

```
Usuario usuario = usuarioAutenticadoService.obtenerUsuarioAutenticado(encabezadoXUserId);
PedidoRespuestaDTO creado = pedidoService.crear(dto, usuario);   // dueño = X-User-Id
```

¿Por qué mitiga el hallazgo? El cliente ya no puede elegir propietario: el campo `usuario` no existe en el contrato de entrada y, si lo intenta enviar, el esquema estricto responde 400.

P1-003 — DTO de registro con rol controlado por el servidor (anti-Mass Assignment)

Antes: el controlador recibía la entidad `Usuario` y Jackson copiaba `"rol":"ADMIN"` (e incluso `"id"`) directamente a la entidad persistida.

Después:

```
// UsuarioRegistroDTO: sin campo "id"; el campo "rol" es WRITE_ONLY:
@JsonProperty(access = JsonProperty.Access.WRITE_ONLY)
private String rol;   // aceptado en el JSON, NUNCA usado: el servidor fija USER

// UsuarioController: el rol SIEMPRE lo asigna el servidor
usuario.setRol("USER");
```

¿Por qué mitiga el hallazgo? La propiedad privilegiada ya no es escribible desde el contrato: aunque el atacante envíe `"rol":"ADMIN"`, el valor se descarta y la cuenta se crea con `USER` (verificado además en la base de datos por `PruebaMassAssignmentTest`).

P1-004 — Minimización de datos: nunca serializar la entidad

Antes: los controladores devolvían entidades completas (`Usuario` con `password`, `Pedido` con el `usuario` embebido).

Después:

```
// Usuario.java
@JsonIgnore
private String password;

// PedidoRespuestaDTO: solo id, descripcion, monto, fecha y nombreUsuario
return new PedidoRespuestaDTO(pedido.getId(), pedido.getDescripcion(),
        pedido.getMonto(), pedido.getFecha(), pedido.getUsuario().getNombre());
```

¿Por qué mitiga el hallazgo? El dato sensible (`password`) queda fuera de toda respuesta por diseño (doble defensa: `@JsonIgnore` + DTOs de salida).

P1-005 — Validación estricta con Bean Validation y errores 400 por campo

Antes: sin `spring-boot-starter-validation`; montos negativos, emails inválidos y contraseñas triviales se aceptaban sin reclamar.

Después:

```
@NotBlank(message = "El nombre es obligatorio")
@Size(min = 3, max = 50, message = "El nombre es obligatorio")
private String nombre;

@Email(message = "El correo electrónico no tiene un formato válido")
private String email;

@Size(min = 8, max = 64, message = "La contraseña debe tener entre 8 y 64 caracteres")
private String password;
```

con `@Valid @RequestBody` en el controlador y `GlobalExceptionHandler` que devuelve 400 con un mensaje por campo (sin stacktrace):

```
@ExceptionHandler(MethodArgumentNotValidException.class)
@ResponseStatus(HttpStatus.BAD_REQUEST)
public Map<String, String> manejarValidacion(MethodArgumentNotValidException ex) { ... }
```

¿Por qué mitiga el hallazgo? La API rechaza entradas malformadas antes de procesarlas y el cliente recibe exactamente qué campo falló, sin información interna del servidor.

Notas de implementación del equipo

1. **Import faltante en el material:** `PedidoController.java` (versión segura) no importaba `com.itson.ciberseguridad.model.Pedido` y no compilaba; se agregó el import.
2. **Mensaje determinista en `nombre`:** para una cadena vacía se violan `@NotBlank` y `@Size` a la vez; ambas restricciones comparten el mismo mensaje para que la respuesta sea determinista (evita que el test sea intermitente).
3. **Siembra de datos sin IDs explícitos:** en `data.sql` se retiraron los IDs manuales porque H2 2.2.224 no avanza la secuencia IDENTITY al insertar valores explícitos (los POST chocaban con la clave primaria). Sin cambio funcional: los datos conservan sus identificadores (Ana=1, Beto=2, pedidos 1–3).

Verificación global (retest)

- [x] `mvn test` corre completo en verde: `Tests run: 9, Failures: 0, Errors: 0` (capturas `evidencias/validacion/08_mvn_test_verde_netbeans.png` y `09_output_test_netbeans.png`).
- [x] Las colecciones Postman de validación muestran las respuestas esperadas (200/403/401/400/201 con rol USER).
- [x] No se introdujeron regresiones funcionales (flujo legítimo 00 responde 200).
- [x] Ningún hallazgo quedó "pendiente" sin justificación escrita.

Hallazgos NO corregidos (si aplica)

ID	Razón documentada	Plan de mitigación
—	No aplica: los 5 hallazgos quedaron corregidos y verificados	—